Lab 6.3: String Processing with Pattern Matching

lab6.3.fsx

```
open System
open System.Numerics
open System.Text.RegularExpressions

let reverseString (s: string) : string =
    String(s.ToCharArray() |> Array.rev)

let isPalindrome (s: string) : bool =
    let cleaned =
        String(s |> Seq.filter (fun c -> Char.IsLetterOrDigit c) |> Seq.map (fun c -> Char.ToLower c)
    cleaned = reverseString cleaned

let transform (s: string) : string =
    match s with
    | "" -> s
    | _ when s |> Seq.forall (fun c -> Char.IsLetter c) -> s.ToUpper()
    | _ when s |> Seq.forall (fun c -> Char.IsDigit c) -> (BigInteger.Parse(s) * 2I).ToString()
    | _ -> s

let emailPattern = @"^[A-Za-z0-9._%+-]+@[A-Za-z0-9-]+(\.[A-Za-z0-9-]+)*\.[A-Za-z]{2,}$"

let (|ValidEmail|InvalidEmail|) (s: string) =
    if Regex.IsMatch(s, emailPattern) then ValidEmail else InvalidEmail

let checkEmail (s: string) : string =
    match s with
    | ValidEmail -> "valid"
    | InvalidEmail -> "invalid"

printfn "=== 1. Reverse and palindrome ==="
[ "madam"; "A man a plan a canal Panama"; "hello" ]
|> List.iter (fun s ->
    printfn "%-30s reversed: %-30s palindrome: %b" s (reverseString s) (isPalindrome s))

printfn "\n=== 2. Transform ==="
[ "hello"; "12345"; "abc123" ]
|> List.iter (fun s -> printfn "%-10s -> %s" s (transform s))

printfn "\n=== 3. Email validation ==="
[ "student@kr.edu"; "john.doe@mail.co.in"; "bad@email"; "no-at-sign.com" ]
|> List.iter (fun s -> printfn "%-22s %s" s (checkEmail s))
```